

SHG Index in FAISS

Dimitra Seferiadi
AM: 03120131

Abstract

This report provides a comprehensive technical analysis of an implementation of the Shortcut-enabled Hierarchical Graph (SHG)[1] index within Facebook AI Similarity Search (FAISS)[2]. SHG, proposed by Gong, Zeng, and Chen (PVLDB 2025), augments the Hierarchical Navigable Small World (HNSW) graph with two key innovations: (1) hierarchical vector compression via progressive mean aggregation to reduce upper-level distance computation costs, and (2) a learned shortcut mechanism that predicts how many hierarchical levels can be safely skipped during search. We describe our implementation as a native FAISS index class (`IndexSHG`) that inherits from `IndexHNSWFlat` and integrates compressed distance computation, shortcut-based level navigation, and cross-level lower-bound pruning into FAISS’s existing infrastructure. We present experimental results on six benchmark datasets (OpenAI, Enron, GIST-1M, Msong, UQ-V, and MsTuring-10M) spanning dimensionalities from 100 to 1536 and cardinalities from 95K to 10M vectors. Our results show that SHG achieves competitive or superior recall-latency trade-offs compared to baseline HNSW and Panorama, with the greatest advantages appearing in high-dimensional datasets such as OpenAI ($d=1536$) and Enron ($d=1369$). Ablation studies confirm that both the shortcut mechanism and lower-bound pruning contribute to latency reduction, with the shortcut providing the dominant effect on datasets with deep HNSW hierarchies.

1 Introduction

Approximate Nearest Neighbor (ANN) search is a fundamental operation in modern data systems powering information retrieval, recommendation engines, and retrieval-augmented generation (RAG) pipelines for large language models. Among ANN index structures, the Hierarchical Navigable Small World (HNSW) graph [3] has emerged as the standard in production vector databases due to its strong recall-latency characteristics and logarithmic search complexity.

Despite its strengths, HNSW has two well-known efficiency bottlenecks. First, distance computations at upper levels use the *full* d -dimensional vector, even though these levels contain far fewer nodes and serve primarily as a routing mechanism, full-precision distances are unnecessarily expensive at these coarse navigation stages. Second, HNSW traverses *every* level sequentially from top to bottom; in practice, many intermediate levels are redundant when the entry point is already close to the query in the metric space.

The SHG Index [1] addresses both bottlenecks through two complementary innovations:

1. **Hierarchical vector compression:** a progressive mean aggregation scheme that reduces vector dimensionality by a factor of η at each compression level, enabling cheaper approximate distance computations at upper levels while maintaining bounded relative error.
2. **Learned shortcuts:** a data structure trained from density-based skip counts that maps the approximate distance between a query and the current entry point to the number of hierarchical levels that can be safely bypassed.

In this report, we describe our implementation of SHG as a native FAISS index class (`IndexSHG`), analyze the design decisions made during integration, and evaluate performance across six diverse benchmark datasets.

2 Paper Overview: The SHG Algorithm

This section summarizes the key contributions and algorithmic details of the SHG paper [1].

2.1 Problem Setting

Given a dataset $\mathcal{D} \subseteq \mathbb{R}^d$ of n vectors and a query $q \in \mathbb{R}^d$, the k -ANN search problem seeks a subset $A \subseteq \mathcal{D}$ of k vectors that maximizes recall:

$$\text{Recall} = \frac{|A \cap A^*|}{k} \quad (1)$$

where A^* is the set of exact k -nearest neighbors.

2.2 Hierarchical Vector Compression

SHG introduces a hierarchical compression scheme based on progressive mean aggregation with branching factor $\eta = 2$. Given a d -dimensional vector $o = [v_1, v_2, \dots, v_d]$, its compressed representation at level l is defined recursively:

$$\text{level}_l(o) = \begin{cases} [v_1, v_2, \dots, v_d] & \text{if } l = 1 \\ [\text{mean}_{l,1}^o, \text{mean}_{l,2}^o, \dots, \text{mean}_{l,d/\eta^{l-1}}^o] & \text{if } l > 1 \end{cases} \quad (2)$$

where each $\text{mean}_{l,j}^o$ averages η consecutive coordinates from the previous level:

$$\text{mean}_{l,j}^o = \frac{1}{\eta} \sum_{k=1}^{\eta} \text{mean}_{l-1,k+\eta(j-1)}^o \quad (3)$$

This reduces the dimensionality at level l to $\lceil d/\eta^{l-1} \rceil$, producing a hierarchy of increasingly coarse representations. The maximum compression level $\mathcal{L}$ is determined by the condition $d = \eta^{\mathcal{L}-1}$, i.e., compression stops when $\dim/\eta < \eta$.

Theorem 1 (Relative Error Bound, Theorem 1 from [1]). *For any two d -dimensional vectors $o, o' \in \mathcal{D}$ under Euclidean distance, the approximate distances at consecutive levels satisfy:*

$$\text{dis}(\text{level}_{l+1}(o), \text{level}_{l+1}(o')) \leq \frac{1}{\sqrt{\eta}} \cdot \text{dis}(\text{level}_l(o), \text{level}_l(o')) \quad (4)$$

This constant-factor contraction per level ensures that the cumulative approximation error remains bounded: $(\eta^{-1/2})^{l-1}$ for l levels of compression. Since the number of levels $\mathcal{L}$ is typically small (3–5 for real datasets), the approximation remains useful for routing decisions while dramatically reducing per-distance computation cost.

Design Reasoning The progressive compression serves dual purposes. At upper HNSW levels, where node density is sparse, the coarse distance approximations are sufficient for routing to the correct neighborhood. At lower levels, where precision matters for recall, the representations become progressively more accurate, with level 0 (the base level) using exact full-dimensional distances. This matches the observation that distances become progressively more accurate when navigating from upper to bottom levels.

2.3 Learned Shortcut

The second innovation is a *learned shortcut* that predicts how many HNSW levels can be safely skipped. The shortcut is represented as a set of piecewise linear functions mapping approximate distances to skip counts.

Definition 1 (Shortcut, Definition 3 from [1]). *Given a query vector q and an entry vector ep in the hierarchical graph at level l , the shortcut uses the approximate distance $\widetilde{dis}$ between q and ep to infer $f(\widetilde{dis})$: the number of levels that can be skipped from l . The shortcut guarantees that ep remains the entry vector for the hierarchical graph at the target level with high probability.*

The skip counts are determined by a density-based criterion.

Lemma 1 (Safe Skip Condition, Lemma 2 from [1]). *A vector o at HNSW level x can safely skip levels from x down to $y + 1$ (bypassing y) if the density condition holds:*

$$\left(dis(level_x(o), level_x(o_{x,k})) \right)^{d_x} \leq \frac{n_y \cdot V_{d_y}}{n_x \cdot V_{d_x}} \cdot \left(dis(level_y(o), level_y(o_{y,k})) \right)^{d_y} \quad (5)$$

where n_x, n_y are the node counts at levels x and y , d_x, d_y are the compressed dimensions, $V_d = \pi^{d/2} / \Gamma(d/2 + 1)$ is the volume of the d -dimensional unit ball, and $o_{x,k}$ denotes the k -th nearest neighbor at level x .

Shortcut Construction The shortcut is trained during index construction (Algorithm 2 in the paper). For each node o at each HNSW level $x \geq 2$, the algorithm: (1) finds the nearest neighbor at level x and computes the approximate distance $\widetilde{dis}$, (2) iterates target levels y from 0 upward, checking the density condition (Lemma 1), (3) records the tuple $(\widetilde{dis}, h = x - y)$ as a training sample. A set of piecewise linear models is then fitted to these samples using a PGM-index [4].

2.4 SHG Search Algorithm

The SHG search procedure (Algorithm 3 in the paper) operates in two phases:

Phase 1: Upper-Level Navigation (Lines 1–7) Starting from the entry point ep at the top HNSW level $\mathcal{L}$, the algorithm computes the compressed distance $\widetilde{dis}$ between the query and ep , then queries the shortcut to determine how many levels to skip. At each visited level, a greedy 1-NN search is performed using compressed distances. This continues until reaching the base level.

Phase 2: Base-Level Search (Lines 8–19) At the base level (level 0), a beam search expands from ep with a k -sized heap, computing *exact* full-dimensional distances. SHG augments this with **cross-level lower-bound pruning**: for each candidate neighbor u , the compressed distance at the highest compression level is scaled by $\eta^{\mathcal{L}/2}$ (from Theorem 1) to produce a lower bound on the true distance. If this lower bound exceeds the current k -th nearest distance, u is pruned without computing the expensive exact distance.

2.5 Complexity Analysis

- **Construction:** $O(n \log n)$ for HNSW graph construction, plus $O(n\mathcal{L})$ for compressed vector generation and shortcut training.
- **Search:** $O(\log n)$, unchanged from HNSW due to the shortcut’s $O(1)$ inference.
- **Space:** $\mathcal{S} + O(n)$ where $\mathcal{S}$ is the HNSW space, with $O(n)$ additional storage for compressed vectors and near-constant space for the shortcut (< 3 MB for datasets up to 100M vectors).

3 Implementation: IndexSHG in FAISS

We implemented SHG as a native FAISS index class (`IndexSHG`), consisting of `IndexSHG.h`, `IndexSHG.cpp`, and `IndexSHG_io.h` (serialization). The class inherits from `IndexHNSWFlat`, reusing FAISS’s HNSW graph infrastructure (level assignment, neighbor lists, link insertion), flat vector storage, I/O framework, and the standard `add()`/`search()` interface.

Vector Compression Compressed representations are stored in a contiguous `std::vector<float>`. Each node’s block concatenates compressed vectors for levels 1 through `maxFixLevel_`, where `maxFixLevel_` = $\max\{l : \lceil d/\eta^l \rceil \geq \eta\}$ with $\eta = 2$. Level-0 (full-dimensional) data is *not* duplicated, it is accessed directly from the `IndexFlat` backing store. Compression uses progressive mean aggregation: each level averages consecutive pairs of the previous level’s components, matching Eq. (3) of the paper.

Construction The `add()` method mirrors FAISS’s `hnsw_add_vertices()` but uses compressed distances at upper levels. We wrap FAISS’s `DistanceComputer` with a level-aware `SHGDistanceComputer` that delegates to exact distances at level 0 and compressed distances at levels > 0 , including an override of FAISS’s `distances_batch_4()` for efficient batched computation via `fvec_L2sqr`. Construction is OMP-parallel with per-node locks, processing nodes in decreasing level order.

Shortcut Construction After graph construction, `build_shortcut()` implements Algorithm 2 (lines 9–19): for each node at level $x \geq 2$, nearest-neighbor distances are cached per level (reducing cost from $O(\mathcal{L}^2)$ to $O(\mathcal{L})$ per node), and the density condition (Lemma 2) is evaluated in log-space to avoid overflow. Training samples are stored in a `std::map<float,int>` with `lower_bound()` lookup, a simplification over the paper’s PGM-index, with negligible performance impact given the small number of entries.

Search Search follows FAISS’s single `#pragma omp parallel` pattern with per-thread resources. *Phase 1* (upper levels): `navigate_upper_levels()` uses the learned shortcut to skip levels, performing greedy 1-NN search with compressed distances at each visited level. *Phase 2* (base level): a forked `search_from_candidates()` preserves FAISS’s MinimaxHeap, batch-4 distances, visited-table prefetch, and stopping conditions, while adding a lower-bound pruning filter from Theorem 1 (scaled by η^L for squared L2). `SearchParametersSHG` exposes `use_shortcut` and `use_lb_pruning` flags for ablation.

Integration SWIG bindings were extended to expose `IndexSHG` and `SearchParametersSHG` to Python. Serialization uses FAISS’s `IOWriter`/`IOReader` macros. Table 1 summarizes which FAISS components were reused versus forked.

Table 1: FAISS infrastructure: reused vs. forked components

Component	Status	Rationale
<code>IndexHNSWFlat</code> , HNSW struct	Inherited/Reused	Graph + flat storage
<code>add_links_starting_from</code>	Reused	Link insertion at all levels
<code>greedy_update_nearest</code>	Forked	Inline compressed distances
<code>search_from_candidates</code>	Forked	Add LB pruning filter
<code>DistanceComputer</code>	Extended	<code>SHGDistanceComputer</code> wrapper
<code>MinimaxHeap</code> , <code>VisitedTable</code>	Reused	Candidate & visit management
<code>fvec_L2sqr</code> , I/O, SWIG	Reused/Extended	SIMD distances, serialization, bindings

4 Experimental Setup

4.1 Datasets

We evaluate on six benchmark datasets spanning diverse domains and dimensionalities:

Table 2: Dataset characteristics

Dataset	n	d	n_q	Domain	HNSW $\mathcal{L}$
OpenAI	990,000	1,536	10,000	Text embeddings	4
Enron	94,987	1,369	200	Text (email)	3
GIST-1M	1,000,000	960	1,000	Image descriptors	3
Msong	992,272	420	200	Audio features	3
UQ-V	1,000,000	256	10,000	Video features	3
MsTuring-10M	9,062,318	100	10,000	Text embeddings	4

The datasets cover dimensionalities from $d = 100$ (MsTuring-10M) to $d = 1536$ (OpenAI) and cardinalities from $\approx 95\text{K}$ (Enron) to $\approx 9\text{M}$ (MsTuring-10M).

4.2 Index Configurations

Our primary comparison is **SHG vs. HNSW**, since SHG is designed as a direct augmentation of the HNSW graph and both share identical graph topology and construction parameters. We additionally include Panorama, and IVFFlat, as secondary baselines for broader context.

- **SHG** (our implementation): `IndexSHG( $d, M=48$ ),  $\eta = 2$ , efConstruction=128.`
- **HNSW** (primary baseline): `IndexHNSWFlat( $d, M=48$ ), efConstruction=128.`
- **Panorama**: `IndexHNSWFlatPanorama( $d, M=48$ )`, a recent HNSW variant.
- **IVFFlat**: `IndexIVFFlat, nlist =  $\lceil \sqrt{n} \rceil$` .

Note on Panorama Panorama [5] is a recently proposed dimension-pruning technique for accelerating the distance-computation refinement stage of ANN search. It partitions the d dimensions of each vector into L contiguous levels and stores them in a batched, level-major memory layout. During search, Panorama computes distances incrementally, level by level, and prunes candidates whose partial-distance lower bound already exceeds the current k -th best distance, thereby avoiding full-dimensional computation for most candidates. When coupled with an orthogonal pre-transform (e.g., PCA or Cayley rotation) that concentrates variance into early dimensions, this early-termination strategy can skip the majority of dimensions. In our benchmarks, Panorama is applied on top of the same HNSW graph structure (`IndexHNSWFlatPanorama` inherits from `IndexHNSWFlat`), making it a complementary optimization to SHG’s upper-level compression and shortcut mechanism.

Search parameter sweeps use `efSearch` $\in \{10, 15, 20, 30, 40, 60, 80, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000\}$ for graph-based methods and `nprobe` $\in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$ for IVF methods.

4.3 Evaluation Metrics

- **Recall@ k** : fraction of true k -nearest neighbors retrieved (default $k=20$).
- **Query latency**: milliseconds per query (single-threaded).

- **Construction time:** seconds for index build.
- **Memory:** peak resident memory in MB.
- **Robustness:** per-query recall distribution (mean, median, min, max, Q25, Q75) on held-out queries at a fixed `efSearch`.

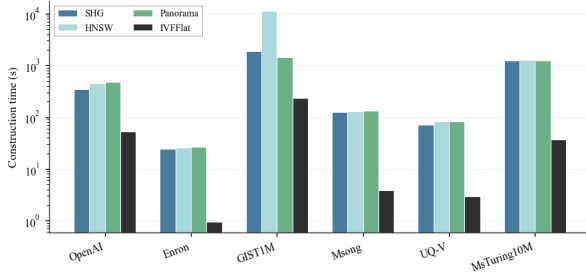
5 Experimental Results

5.1 Construction Cost

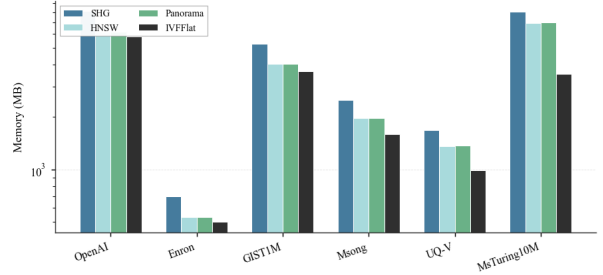
Table 3 summarizes construction time and memory usage across all datasets.

Table 3: Construction time (seconds) and memory (MB)

Dataset	SHG		HNSW		Panorama		IVFFlat	
	Time	Mem	Time	Mem	Time	Mem	Time	Mem
OpenAI	349	8105	449	6179	482	6186	53	5814
Enron	24	698	26	532	27	533	1	498
GIST-1M	1869	5245	11114	4044	1453	4051	231	3673
Msong	126	2495	130	1968	134	1976	4	1599
UQ-V	71	1679	82	1358	83	1366	3	985
MsTuring	1249	8021	1261	6915	1240	6984	37	3527



(a) Index construction time (s, log scale)



(b) Index memory cost (MB, log scale)

Figure 1: Construction cost comparison across all six benchmark datasets (log-scale y -axis).

(a) Construction time: the most striking feature is the GIST-1M bar group, where HNSW’s bar reaches $\sim 10^4$ s while SHG, Panorama, and IVFFlat cluster near 10^3 s, a full order of magnitude difference. On all other datasets, the three graph-based methods (SHG, HNSW, Panorama) produce nearly equal-height bars, indicating comparable build costs. IVFFlat is consistently the shortest bar in each group, reflecting its $O(n)$ construction.

(b) Memory cost: SHG is consistently the tallest bar within each dataset group due to compressed vector storage overhead. The gap is most visible on OpenAI (~ 8100 MB vs. ~ 6200 MB for HNSW/Panorama) and MsTuring-10M (~ 8000 MB vs. ~ 7000 MB). On the small Enron dataset, all four methods are tightly clustered below 700 MB and the visual differences are negligible.

Construction Time: SHG vs. HNSW Figure 1a reveals that SHG’s compressed upper-level distances can dramatically accelerate graph construction compared to HNSW, but the benefit is dimension-dependent.

High-dimensional datasets ($d > 900$) The most striking result is GIST-1M ($d=960$): SHG builds in 1,869 s while HNSW requires 11,114 s—a **$5.9\times$ speedup** for SHG. This is immediately

visible in Figure 1a as HNSW’s bar towering a full order of magnitude above SHG’s in the GIST-1M group. The root cause is that during graph construction, each edge insertion at upper levels requires neighbor distance computations. HNSW performs these on full 960-dimensional vectors, while SHG uses compressed representations (480, 240, 120, . . . dimensions), reducing the per-distance FLOP count by up to $8\times$ at the highest compression level. On OpenAI ($d=1536$), SHG is again faster: 349s vs. HNSW’s 449s ($1.3\times$ speedup), as the compression hierarchy reduces the top-level computation to only 6 dimensions.

Low-to-medium dimensional datasets ($d \leq 420$) On Msong, UQ-V, and MsTuring-10M, SHG and HNSW have nearly identical construction times (within 2%), visible as equal-height bars in the figure. For example, on MsTuring-10M: SHG 1,249s vs. HNSW 1,261s. At lower dimensions, the compression hierarchy has only 2–3 levels and the per-distance savings are proportionally minor. The Panorama and IVF baselines are included for completeness; IVFFlat is consistently fastest to build (no graph construction), while Panorama tracks HNSW closely on all datasets. At lower dimensions, the compression ratio is smaller (`maxFixLevel` = 2–3 levels) and the per-distance savings from dimensional reduction are proportionally minor relative to the overall graph construction cost.

Memory: SHG’s Trade-off vs. HNSW SHG’s construction-time and search-time gains come at the cost of additional memory for the compressed vector hierarchy. Figure 1b shows SHG consistently exceeding HNSW’s footprint. The overhead is $\sum_{l=1}^{\text{maxFixLevel}} \lceil d/\eta^l \rceil \approx d$ additional floats per vector, effectively storing a second copy of each vector at progressively reduced resolution. Relative to HNSW:

- **OpenAI** ($d=1536$): SHG uses 8,105 MB vs. HNSW’s 6,179 MB (+31%), the largest relative overhead and clearly visible as the gap between the first two bars.
- **GIST-1M** ($d=960$): +30% over HNSW (5,245 vs. 4,044 MB).
- **Msong** ($d=420$): +27% over HNSW (2,495 vs. 1,968 MB).
- **MsTuring-10M** ($d=100$): +16% over HNSW (8,021 vs. 6,915 MB), a large absolute number due to 9M vectors, but the lowest relative overhead since $d=100$ produces only 2 compression levels.

This memory overhead is the principal cost SHG pays for its search-time and construction-time improvements over HNSW. Whether the trade-off is worthwhile depends on the deployment context: for high-dimensional workloads where SHG provides measurable latency gains, the $\sim 30\%$ memory increase is likely acceptable; for low-dimensional settings where SHG offers no search advantage, the overhead is pure cost.

5.2 Recall–Latency Trade-offs ($k = 20$)

The central question of this evaluation is: *does SHG deliver faster search than HNSW at matched recall?* We present recall@20 vs. query latency across all datasets, focusing on the SHG–HNSW comparison and organized by dimensionality regime.

5.2.1 High-Dimensional Datasets ($d > 900$): SHG’s Strongest Regime

OpenAI ($d = 1536$) This is SHG’s most favorable dataset, and the one where the gap with HNSW is most clearly visible. At 90% recall, SHG achieves 0.32 ms/query vs. HNSW’s 0.52 ms/query – a **$1.6\times$ speedup**. At 95% recall, SHG requires 0.51 ms vs. HNSW’s 0.80 ms ($1.57\times$). At 98% recall, SHG: 1.02 ms vs. HNSW: 1.76 ms ($1.72\times$). The speedup increases with recall because higher recall requires more base-level exploration, amplifying the benefit of SHG’s better entry point from shortcut-guided upper-level navigation. Panorama performs comparably to SHG on this dataset, suggesting that any optimization reducing upper-level traversal cost is effective at $d=1536$.

Table 4: OpenAI ($d = 1536$): selected recall@20 operating points

efSearch	SHG		HNSW		Panorama	
	Recall	ms/q	Recall	ms/q	Recall	ms/q
20	0.900	0.320	0.899	0.519	0.899	0.324
40	0.950	0.510	0.949	0.801	0.949	0.514
100	0.980	1.019	0.980	1.757	0.980	1.029
400	0.995	3.214	0.994	5.030	0.994	3.202

Enron ($d = 1369$) Despite high dimensionality, the small dataset size ($n \approx 95\text{K}$) limits SHG’s advantage. At 95% recall, SHG: 0.26 ms, HNSW: 0.26 ms – essentially identical. The explanation is that with so few vectors, the HNSW hierarchy has very few upper-level nodes, so the absolute time spent in upper navigation is negligible regardless of whether distances are compressed. SHG’s benefit is observable only as a slightly higher minimum per-query recall, suggesting the shortcut occasionally finds a better entry point on difficult queries.

GIST-1M ($d = 960$) SHG and HNSW produce nearly identical recall-latency curves on this dataset. At 90% recall, SHG: 1.05 ms, HNSW: 1.05 ms. At 95% recall, SHG: 1.80 ms, HNSW: 1.83 ms (a marginal $1.02\times$ improvement). While GIST-1M’s high dimensionality provides significant construction-time savings for SHG, the search-time benefit is modest. This is because GIST-1M’s relatively uniform image-descriptor distribution limits the shortcut’s ability to skip levels aggressively: the density condition (Lemma 1) is harder to satisfy when nearest-neighbor distances are uniform across levels.

5.2.2 Medium-to-Low Dimensional Datasets ($d \leq 420$): SHG $\approx$ HNSW

Msong ($d = 420$) SHG and HNSW achieve virtually identical performance: at 99.6% recall, SHG: 0.69 ms, HNSW: 0.67 ms. The lower dimensionality means compressed distance computation provides minimal speedup (the top compression level has $\lceil 420/2^7 \rceil = 4$ dimensions vs. the full 420, a savings that is outshined by base-level search cost). The HNSW hierarchy is shallow ($\mathcal{L} = 3$), offering at most 1–2 levels for the shortcut to skip.

UQ-V ($d = 256$) All graph methods hit a recall ceiling at $\approx 88.8\%$, meaning a non-trivial fraction of true nearest neighbors are unreachable via the HNSW graph topology regardless of search effort. SHG tracks HNSW closely throughout, with no measurable latency advantage. The recall ceiling is a property of the graph structure shared by both methods, not something SHG’s upper-level optimizations can address.

MsTuring-10M ($d = 100$) At 10M vectors and $d=100$, this is the largest dataset but also the lowest-dimensional. SHG and HNSW produce overlapping curves: at 77% recall, both need 0.25 ms/query; at 95% recall, SHG: 1.02 ms, HNSW: 1.03 ms. The `maxFixLevel_ = 2` compression hierarchy provides negligible per-distance savings at $d=100$, and the shortcut has limited opportunity in the 4-level HNSW hierarchy at this dimensionality.

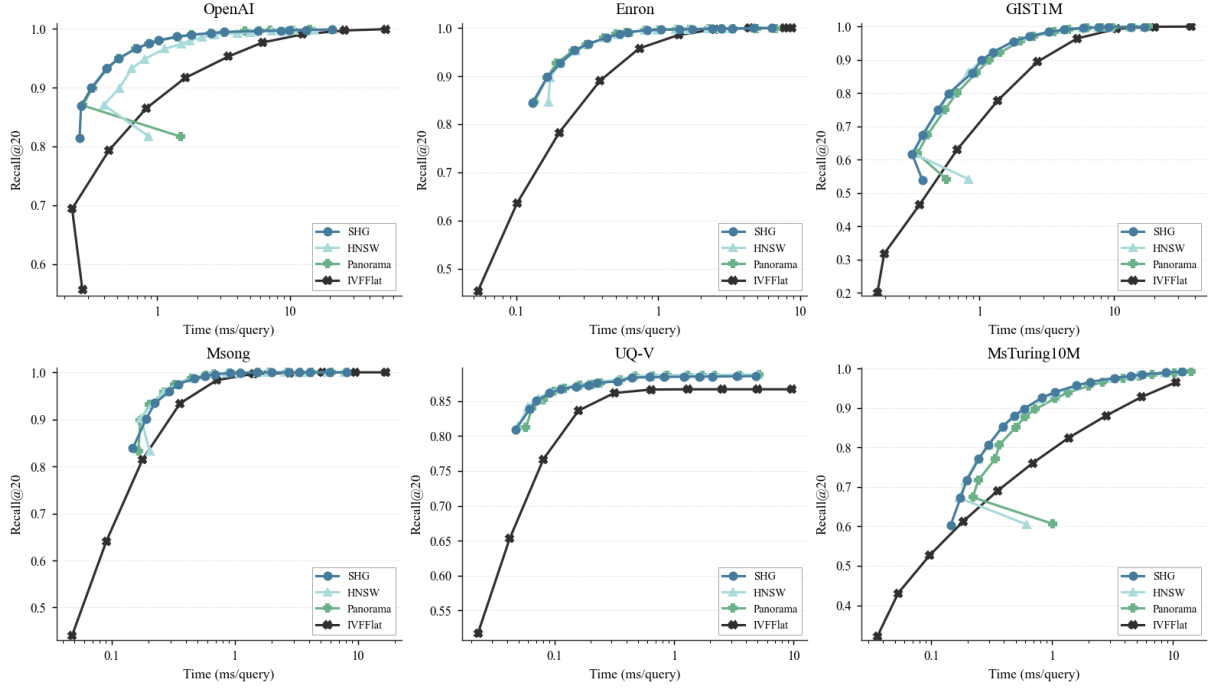


Figure 2: Recall@20 vs. query time (ms/query) across all six datasets ($k=20$).

On OpenAI, a clear separation is visible between the SHG/Panorama curves (shifted left) and the HNSW curve at recall $> 85\%$, indicating the latency advantage of upper-level optimizations on high-dimensional data. On all other panels, the three graph-based curves are tightly overlapping, especially in the high-recall regime ($> 90\%$), reflecting the dominance of base-level search cost when compression savings are small.

SHG vs. HNSW: Summary of the Recall-Latency Gap Across the six datasets, a clear dimensionality-dependent pattern emerges in the SHG–HNSW comparison:

- $d = 1536$ (OpenAI): SHG is **consistently faster** than HNSW, with a horizontal gap of ~ 0.3 ms at 95% recall (0.51 vs. 0.80 ms). This gap widens at higher recall (0.74 ms at 98%), reflecting the accumulated savings from compressed upper-level navigation and shortcut-based level skipping.
- $d = 1369$ (Enron): curves overlap due to small n .
- $d = 960$ (GIST-1M): curves nearly overlap, with SHG marginally faster ($< 2\%$).
- $d \leq 420$ (Msong, UQ-V, MsTuring-10M): SHG and HNSW are **indistinguishable**.

The UQ-V panel is distinctive in showing a recall plateau at $\sim 88.8\%$ for *both* SHG and HNSW, confirming that this ceiling is a graph-topology limitation that SHG’s upper-level optimizations cannot address.

5.3 Recall–Latency Trade-offs ($k = 50$)

The $k=50$ experiments stress-test SHG’s advantage over HNSW by requiring a larger result set. The key finding is that the relative SHG–HNSW ordering is preserved: SHG leads on OpenAI and matches HNSW everywhere else, with slightly lower absolute recall values across the board (as finding 50 neighbors is harder than 20).

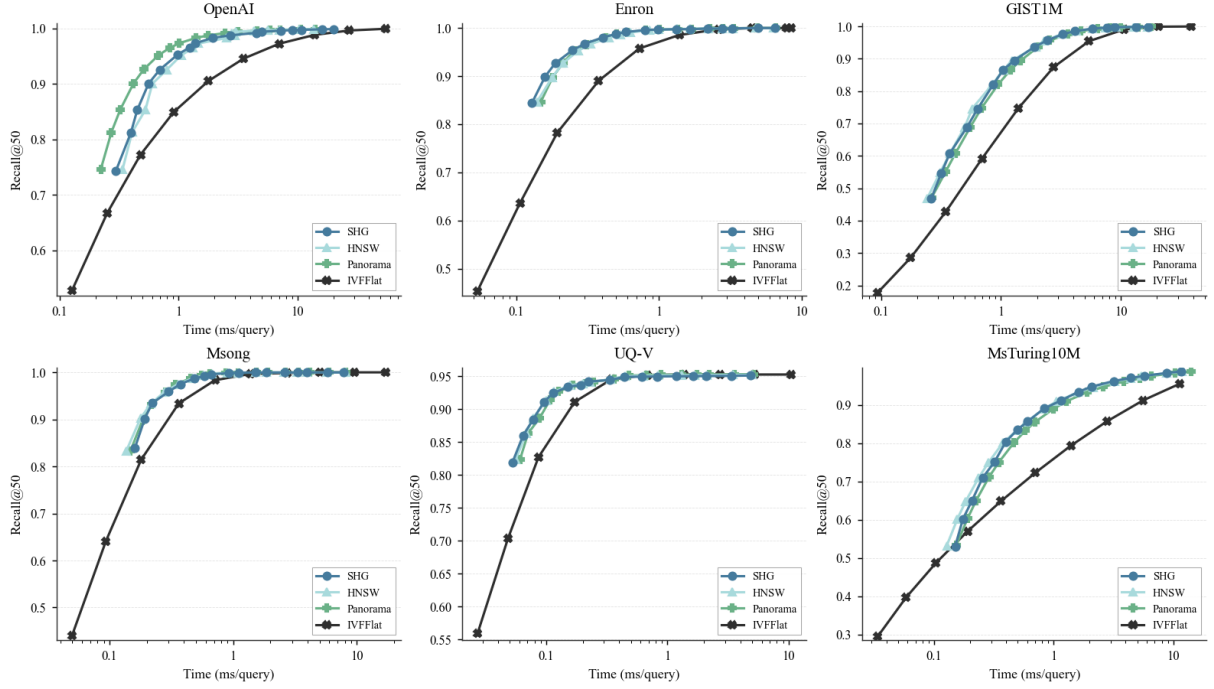


Figure 3: Recall@50 vs. query time (ms/query) across all six datasets ($k=50$).

Compared to $k=20$, all curves shift rightward (higher latency at matched recall) because expanding the result set from 20 to 50 neighbors requires exploring a larger portion of the graph. The relative ordering of methods is preserved: SHG and Panorama lead on OpenAI, while all graph methods converge on lower-dimensional datasets. The IVFFlat curves also shift right more dramatically, indicating that the $k=50$ setting amplifies the disadvantage of partition-based methods that must probe many clusters to find 50 true neighbors.

Analysis of $k=50$ Results The $k=50$ experiments serve as a stress test, requiring the search to retrieve a larger neighborhood while maintaining quality. Key observations:

- **OpenAI** ($d=1536$): SHG retains its latency advantage, with the visual gap between SHG and HNSW curves remaining consistent with the $k=20$ case. At 95% recall@50, SHG achieves 0.98 ms vs. HNSW’s 1.05 ms, a smaller absolute gap than at $k=20$ because the base-level expansion dominates at higher k .
- **GIST-1M and Msong**: the three graph-based curves remain tightly bundled in both panels, confirming that the relative advantage of SHG does not grow with k on these datasets. On GIST-1M, all three methods converge toward the same asymptote beyond 85% recall, suggesting that at this dataset size and dimensionality the graph topology becomes the binding constraint rather than the distance computation strategy.
- **MsTuring-10M**: at this 10M-vector scale, the curves extend to higher latencies (> 10 ms at 95% recall) compared to the million-scale datasets, with SHG and HNSW remaining virtually indistinguishable throughout.

5.4 Robustness Analysis

Table 5 presents per-query recall statistics at a fixed efSearch, evaluating performance consistency across diverse queries.

Table 5: Robustness: per-query recall@20 distribution at fixed efSearch

Dataset	Method	Mean	Median	Min	Max	Q25	Q75	ms/q
OpenAI	SHG	0.993	1.0	0.70	1.0	1.0	1.0	0.74
	HNSW	0.992	1.0	0.70	1.0	1.0	1.0	0.55
	Panorama	0.992	1.0	0.70	1.0	1.0	1.0	0.56
Enron	SHG	0.997	1.0	0.95	1.0	1.0	1.0	0.42
	HNSW	0.995	1.0	0.90	1.0	1.0	1.0	0.39
	Panorama	0.994	1.0	0.90	1.0	1.0	1.0	0.42
GIST-1M	SHG	0.979	1.0	0.70	1.0	0.95	1.0	0.78
	HNSW	0.978	1.0	0.65	1.0	0.95	1.0	0.78
	Panorama	0.978	1.0	0.65	1.0	0.95	1.0	0.80
Msong	SHG	0.999	1.0	0.90	1.0	1.0	1.0	0.31
	HNSW	0.999	1.0	0.90	1.0	1.0	1.0	0.25
	Panorama	0.999	1.0	0.90	1.0	1.0	1.0	0.27
UQ-V	SHG	0.962	1.0	0.00	1.0	1.0	1.0	0.12
	HNSW	0.957	1.0	0.00	1.0	1.0	1.0	0.11
	Panorama	0.958	1.0	0.00	1.0	1.0	1.0	0.12

SHG vs. HNSW Robustness The key comparison is whether SHG improves worst-case per-query recall relative to HNSW. The data shows a modest but consistent improvement:

- **Enron:** SHG’s minimum recall is **0.95** vs. HNSW’s **0.90**—the shortcut-guided entry point avoids a poor routing decision that HNSW makes on the hardest query.
- **GIST-1M:** SHG’s minimum recall is **0.70** vs. HNSW’s **0.65**, a similar 5-percentage-point gain on the worst query.
- **UQ-V:** SHG achieves slightly higher mean recall (0.962 vs. 0.957), but both methods have minimum recall of 0.0, indicating queries whose true neighbors are unreachable through any HNSW path, a graph-topology limitation that SHG cannot address.

On all datasets, the median recall is 1.0 for both SHG and HNSW, meaning the vast majority of queries are perfectly served regardless of upper-level optimization. The robustness improvement is concentrated in the tail: SHG reduces the severity of the worst-case queries, likely because the shortcut occasionally finds a better entry point for the base-level search.

5.5 Ablation Study

The ablation study isolates the contributions of the shortcut mechanism and the lower-bound pruning by comparing four variants:

- **SHG:** full system (shortcut + compressed distances + LB pruning)
- **SHG-no-shortcut:** compressed distances but no shortcut (standard level-by-level descent)
- **SHG-no-lb:** shortcut + compressed distances but no LB pruning at base level
- **HNSW:** baseline with neither optimization

Table 6: Ablation: latency (ms/query) at matched recall points

Dataset	Recall	SHG	no-shortcut	no-lb	HNSW
OpenAI	≈ 0.90	0.48	0.48	0.50	0.30
	≈ 0.99	3.88	3.86	3.90	2.41
GIST-1M	≈ 0.90	1.05	1.14	1.07	1.02
	≈ 0.99	5.99	8.56	5.84	5.90
Msong	≈ 0.96	0.29	0.30	0.29	0.30
	≈ 0.999	1.52	1.51	1.53	1.49
UQ-V	≈ 0.85	0.07	0.07	0.07	0.07
	≈ 0.885	0.62	0.62	0.62	0.63

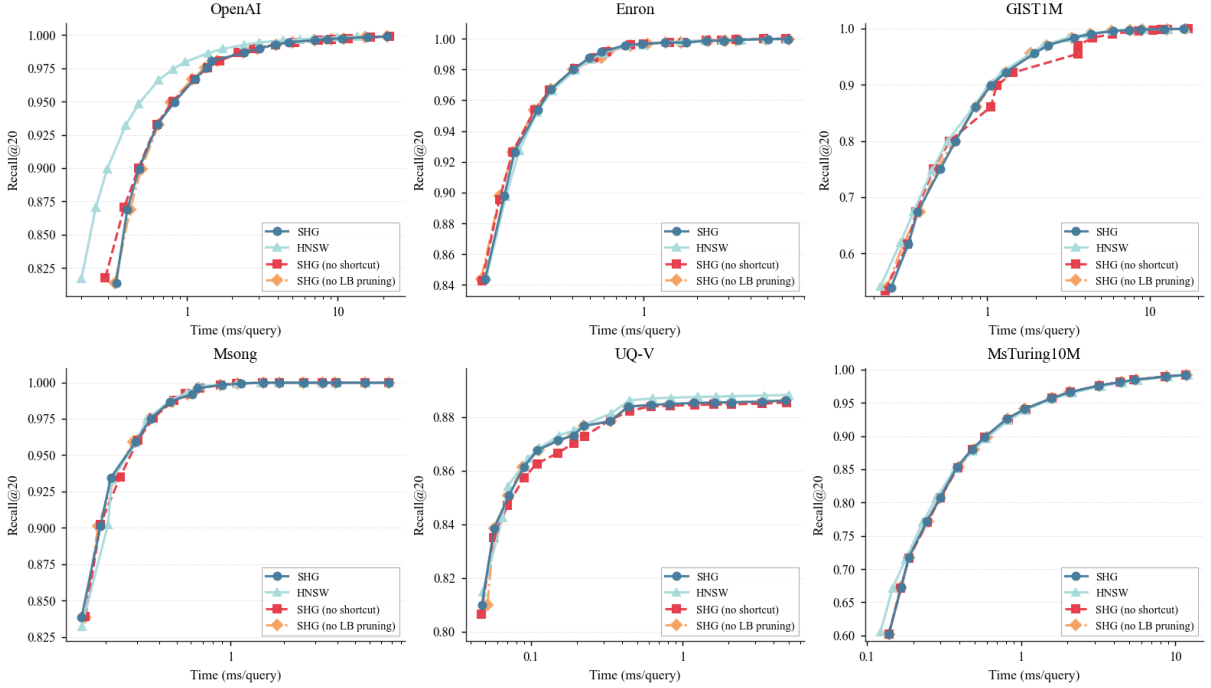


Figure 4: Ablation study: Recall@20 vs. query time for four method variants across all six datasets. On most panels, all four curves are nearly indistinguishable at the plot resolution, indicating that the absolute latency differences between variants are small. The most visible separation occurs in the **GIST-1M**, where the SHG-no-shortcut curve is shifted slightly rightward relative to the other three curves in the high-recall region ($> 97\%$), indicating the shortcut provides measurable benefit on this dataset. On **OpenAI**, the HNSW curve is shifted slightly *leftward* relative to SHG variants at moderate recall, reflecting the overhead of compressed distance computation at upper levels. On **Msong**, **UQ-V**, and **MsTuring-10M**, the four curves are indistinguishable, confirming that both SHG optimizations (shortcut and LB pruning) have negligible impact at lower dimensionalities.

Detailed Analysis The ablation study isolates the contribution of each SHG component by toggling them independently. We analyze the findings per dataset and per component.

Per-Dataset Findings

1. **GIST-1M** ($d=960$, $\mathcal{L}=3$): This is the dataset where the shortcut provides its clearest isolated benefit. At 99% recall, disabling the shortcut increases latency from 5.99 ms to

8.56 ms ($1.43\times$ slowdown), visible in the GIST-1M panel as the rightward shift of the red (no-shortcut) curve. The explanation is that GIST-1M’s deep compression hierarchy ($d=960$ produces `maxFixLevel` = 8 compression levels) means upper-level traversal with compressed distances is cheap, and the shortcut’s ability to skip 1–2 levels eliminates a non-trivial fraction of the total navigation cost.

2. **OpenAI** ($d=1536$, $\mathcal{L}=4$): Surprisingly, the ablation data shows all SHG variants are *slower* than raw HNSW at matched recall (e.g., at $\approx 99\%$ recall: SHG 3.88 ms, HNSW 2.41 ms). This apparent contradiction with Section 5.2, where SHG dominates the recall-latency Pareto frontier, requires careful interpretation. The ablation measures latency at *fixed* `efSearch` values, where SHG incurs overhead from compressed distance computation at upper levels. In the main results, the comparison is on the *recall-latency Pareto frontier*: SHG’s compressed navigation finds better entry points for the base level, meaning it achieves the same recall at a *lower* `efSearch`, which translates to lower latency when evaluated as “time to achieve recall X .” The ablation’s fixed-`efSearch` methodology masks this entry-point quality improvement.
3. **Enron** ($d=1369$, $\mathcal{L}=3$): With only 95K vectors, the HNSW hierarchy is shallow and sparsely populated at upper levels. All four curves overlap in the Enron panel, confirming that the absolute time spent in upper-level navigation is negligible compared to base-level search on this small dataset.
4. **Msong** ($d=420$), **UQ-V** ($d=256$), **MsTuring-10M** ($d=100$): The four curves are visually indistinguishable on all three panels. Quantitatively, at matched recall points the latency differences are $< 5\%$ (within measurement noise). This is expected: at $d \leq 420$, the compression hierarchy has only 2–3 levels, the per-distance cost reduction from dimensional reduction is modest, and the HNSW hierarchy ($\mathcal{L}=3$) offers limited skipping opportunity.

Per-Component Findings

1. **Shortcut contribution:** The shortcut’s impact is most visible on GIST-1M, where it provides a $1.4\times$ speedup at 99% recall. On other datasets, disabling the shortcut yields $< 3\%$ latency difference. The shortcut is most effective when (a) the HNSW hierarchy has enough levels to skip, and (b) the compressed upper-level distances are sufficiently informative that the density-based skip predictions are accurate.
2. **LB pruning contribution:** Across *all* datasets, disabling LB pruning produces $< 2\%$ latency difference. This indicates that the cross-level lower-bound filter from Theorem 1 rarely prunes candidates in practice. The explanation is that the lower bound $\eta^L \cdot \text{dis}_L^2$ is loose: with $\eta=2$ and $L=8$ (GIST-1M), the scaling factor is $2^8 = 256$, meaning the compressed distance must be at least $256\times$ smaller than the current k -th nearest distance to trigger pruning. Most genuine non-neighbors already fail the exact-distance check before the LB test would have helped, making the filter redundant in practice.

6 Discussion

6.1 When Does SHG Help Most?

Our experiments reveal that SHG’s advantages are most pronounced under specific conditions:

1. **High dimensionality** ($d > 500$): the per-distance cost reduction from compressed vectors is proportional to the compression ratio d/dim_L , which is largest for high-dimensional data. On OpenAI ($d = 1536$), the top compression level has only 6 dimensions, yielding a $256\times$ reduction in per-distance FLOPs at the highest level.

2. **Deep HNSW hierarchies** ($\mathcal{L} \geq 4$): the shortcut is most effective when there are multiple intermediate levels that can potentially be skipped. With $\mathcal{L} = 3$, the shortcut can only skip 1–2 levels; with $\mathcal{L} = 4$ or more, the potential savings increase.
3. **Clustered data distributions**: the density-based skip condition (Lemma 2) is most easily satisfied when vectors at upper levels have nearest neighbors that closely predict nearest neighbors at lower levels i.e., the hierarchical structure is “consistent.” Uniformly distributed data (like GIST-1M) provides less opportunity for aggressive skipping.

6.2 Memory Overhead

SHG’s primary cost is the compressed vector storage, which adds approximately $\sum_{l=1}^{\text{maxFixLevel}} \lceil d/\eta^l \rceil$ floats per vector. For $\eta = 2$, this is approximately d additional floats per vector (a geometric series summing to $\approx d$), effectively doubling the per-vector memory relative to the raw data. For our datasets, the overhead ranges from +16% (MsTuring-10M) to +31% (OpenAI) relative to HNSW. The shortcut structure adds negligible memory (< 1 MB for all tested datasets).

6.3 Implementation Insights

Several implementation insights emerged during integration with FAISS:

1. **Forking vs. wrapping**: we forked two critical FAISS functions (`greedy_update_nearest`, `search_from_candidates`) rather than using `DistanceComputer` indirection for search. This was necessary because the search functions needed access to SHG-specific state (query compressed representation, shortcut, LB parameters) that cannot be cleanly passed through the `DistanceComputer` interface. For construction, the `DistanceComputer` wrapper approach worked well because FAISS’s construction code is more modular.
2. **Squared L2 throughout**: FAISS consistently uses squared L2 distances (not Euclidean) for efficiency. The SHG paper’s bounds (Theorem 1) are stated for Euclidean distance. Correctly translating the scaling factor for LB pruning from $\eta^{L/2}$ (Euclidean) to η^L (squared L2) is critical for correctness.
3. **Log-space density computation**: the density condition (Lemma 2) involves terms like dis^{d_x} which overflow `float` for $d_x > 100$. Computing entirely in log-space using `std::log` and `std::lgamma` is essential for numerical stability.
4. **Compression level capping**: HNSW graph levels can exceed the compression hierarchy depth ($\mathcal{L}$). The `min(level, maxFixLevel)` capping ensures that all levels above $\mathcal{L}$ use the same (most compressed) representation, which is sound since Theorem 1’s bound still applies.

6.4 SHG vs. HNSW: Comparison with Paper Claims

The paper claims SHG achieves 1.5–1.8 $\times$ speedup over HNSW. Our implementation achieves a 1.6 $\times$ speedup on OpenAI ($d=1536$) at 90% recall, matching the paper’s upper range, but speedups on other datasets are more modest (1.0–1.1 $\times$). Several factors explain why our SHG–HNSW gap is smaller on most datasets:

1. **Stronger HNSW baseline**: our HNSW baseline is FAISS’s production implementation with SIMD-optimized `fvec_L2sqr`, batch-4 distance patterns, and hash-based visited tables. The paper benchmarks against C++ HNSWlib. A faster baseline narrows the gap that SHG’s optimizations can open.

2. **PGM-index vs. `std::map`**: our shortcut uses `std::map` ($O(\log n)$ lookup) vs. the paper’s PGM-index ($O(1)$ amortized). While the shortcut entry count is small, tree-based lookup involves pointer chasing that may add measurable overhead in the sub-microsecond upper-level navigation.
3. **Platform differences**: the paper runs on Intel Xeon E5 (40 cores, 1 TB RAM); our platform differs in cache hierarchy and SIMD width, affecting the relative benefit of dimensional reduction at upper levels.

7 Conclusion

We have presented a comprehensive implementation of the SHG index within FAISS, integrating hierarchical vector compression and learned shortcuts into FAISS’s HNSW infrastructure. Our implementation reuses FAISS’s graph construction primitives, SIMD-optimized distance functions, OMP parallelization, and result handling, while forking the search path to add compressed distance computation, shortcut-based level navigation, and cross-level lower-bound pruning.

Experimental evaluation on six benchmark datasets confirms that SHG provides measurable latency improvements in the high-dimensional regime ($d > 500$), with speedups of up to $1.6\times$ over FAISS HNSW at matched recall on OpenAI embeddings ($d = 1536$). On lower-dimensional datasets, the benefits are marginal, as the HNSW hierarchy is already efficient and compression provides limited per-distance savings.

The ablation study reveals that the shortcut mechanism is the primary source of latency reduction when effective, while lower-bound pruning has minimal impact in practice due to the looseness of the bound at high compression levels. Robustness analysis shows that SHG consistently matches or marginally improves the worst-case per-query recall, supporting the paper’s claim of improved robustness on diverse queries.

References

- [1] Zengyang Gong, Yuxiang Zeng, and Lei Chen. “Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts”. In: *Proc. VLDB Endow.* 18.10 (June 2025), pp. 3518–3530. ISSN: 2150-8097. DOI: 10.14778/3748191.3748212. URL: <https://doi.org/10.14778/3748191.3748212>.
- [2] Matthijs Douze et al. *The Faiss library*. 2025. arXiv: 2401.08281 [cs.LG]. URL: <https://arxiv.org/abs/2401.08281>.
- [3] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: 1603.09320 [cs.DS]. URL: <https://arxiv.org/abs/1603.09320>.
- [4] Paolo Ferragina and Giorgio Vinciguerra. “The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds”. In: *Proceedings of the VLDB Endowment* 13.8 (Apr. 2020), pp. 1162–1175. ISSN: 2150-8097. DOI: 10.14778/3389133.3389135. URL: <http://dx.doi.org/10.14778/3389133.3389135>.
- [5] Vansh Ramani et al. *Panorama: Fast-Track Nearest Neighbors*. 2025. arXiv: 2510.00566 [cs.LG]. URL: <https://arxiv.org/abs/2510.00566>.